



GLOSSÁRIO SQL

SQL: UTILIZANDO OPERADORES LÓGICAS

// Glossário SQL_

AND, OR, NOT E BETWEEN

Os filtros compostos nos ajudam a detalhar ainda mais nossas consultas e trazer mais especificações ao mesmo tempo. Por exemplo, como podemos fazer caso a gente queira saber apenas os pedidos com valor **maior ou igual a 200 reais** e com o **status pendente**?

```
SELECT * FROM tabelapedidos  
WHERE total_do_pedido >= 200 AND status =  
'Pendente';
```



Então, o **AND** adiciona mais uma condição ao nosso filtro, assim conseguimos filtrar utilizando essas duas especificações ao mesmo tempo.

Já o **OR** adicionarmos quando especificamos que queremos filtrar os dados que cumpram a uma especificação ou a outra.

Por exemplo, vamos filtrar todos os pedidos que ainda não foram enviados, ou seja, que estejam com o status **pendente** ou **processando**:

```
SELECT * FROM tabelapedidos WHERE  
status = 'Pendente' OR status = 'Processando';
```

Todos os pedidos com esses dois tipos de status foram **filtrados**.

Agora, se eu acrescentar o **NOT** após o **WHERE** ele irá me retornar todos os pedidos menos os pedidos com o status pendente.

```
SELECT * FROM tabelapedidos  
WHERE NOT status = 'Pendente';
```

Também temos a função **BETWEEN**, a tradução seria "entre", então utilizamos ela quando queremos selecionar um intervalo inteiro, vamos ver na prática:

```
SELECT * FROM tabelapedidos  
WHERE data_de_envio_estimada  
BETWEEN '2023-08-01' AND '2023-09-01';
```

Dessa forma serão filtrados todos os pedidos que foram feitos nesse intervalo de tempo, ou seja, que estão **entre** o dia 01/08/23 e 01/09/23.

// Glossário SQL_

IN E DISTINCT



A cláusula **IN** é usada em SQL para verificar se um valor corresponde a qualquer valor em uma **lista específica** de valores.

Por exemplo, vamos retornar os **nomes** dos clientes que fizeram pedidos no mês de **janeiro**

```
SELECT Nome
FROM clientes
WHERE ID IN (
    SELECT ID_Cliente
    FROM pedidos
    WHERE strftime('%m', DataHoraPedido) = '01'
);
```

Já o filtro **DISTINCT**, traduzindo significa "distinto", com esse filtro conseguimos trazer apenas os dados distintos de uma certa coluna, por exemplo, para consultar apenas o número de clientes únicos contidos na base de dados vamos usar o DISTINCT da seguinte forma:

```
SELECT DISTINCT ID_Cliente_pedidos  
FROM tabelapedidos;
```

UTILIZE E DOMINE O SQL!

Parabéns por explorar o **Glossário de SQL**! Agora que você adquiriu os comandos mais avançados da linguagem, é hora de aplicar esse conhecimento. Utilize este material como referência em seus projetos e desafios, praticando para aprimorar suas habilidades na manipulação de bancos de dados. Ao se tornar mais confiante na linguagem SQL, você estará preparado para **enfrentar novos desafios**.

Muito obrigado por chegar até aqui e nos vemos nos próximos cursos da formação em SQL da Alura. Até mais!

AVALIE O CURSO E DEIXE UM COMENTÁRIO.

Compartilhe um resumo de seus novos conhecimentos em suas redes sociais.

alura



Escola Data Science